LoraTrack

Technical Documentation and User Guide

Document version: 1.0

Classification: Public product documentation

Document Control

Field	Value
Product	LoraTrack
Document type	Technical Documentation and User Guide
Document version	1.0
Audience	Users, administrators, engineering, operations, and security teams

> This documentation describes product capabilities and procedures. References to practices or standards do not constitute certification, independent assurance, or formal customer acceptance.

Document Index

LoraTrack User Guide

Purpose and Audience

This guide describes the functional use of LoraTrack for administrators, supervisors, operators, engineering personnel, and read-only users. Available options depend on the user role and active organization.

Access and Sessions

1. Open the HTTPS URL provided by the administrator.
2. Enter the email address and password associated with your account, or use Microsoft sign-in when enabled by the organization.
3. When credentials are invalid, the application displays a generic message and does not disclose whether the email address is registered.
4. Sign out when work is complete, particularly on shared workstations.

Access is always restricted to the active organization. Users who belong to multiple organizations must verify the selected context before viewing or modifying information.

Primary Navigation

- **Dashboard:** operational status, recent activity, locations, and alerts.
- **Products:** commercial definitions and references received from external catalogs.
- **Assets:** trackable physical instances, status, mobility, and assigned devices.
- **Devices:** registered beacons, scanners, gateways, and trackers.
- **Locations:** sites, buildings, floors, zones, floor plans, and known installations.
- **Connectors:** telemetry and catalog integrations, subject to authorization.
- **Users and settings:** memberships, roles, visual identity, and administration.

Products and Assets

A product is a catalog definition; an asset is an individual physical unit. A SKU must not be used as the unique identifier of an asset.

When creating or editing an asset:

1. Select the applicable product.
2. Enter a unique asset tag and a recognizable name.
3. Define its mobility behavior.
4. Assign a compatible device and tracking strategy.
5. Verify the assignment start date.

Assignments are historical records. To replace a device, close the current assignment and create a new one without altering prior records.

Devices and Locations

Devices must be registered with their actual technical identifier. Fixed scanners and beacons require an active installation associated with a location or floor plan. Before relying on a position:

- confirm that the device is active;
- confirm that its installation belongs to the correct floor;
- verify the floor plan scale and physical dimensions;
- validate calibration, reference RSSI, and path-loss exponent where applicable.

Floor Plans and Zones

Floor plans are stored privately. To configure a floor plan:

1. Upload the supported source file and preview.
2. Specify its physical dimensions in meters.
3. Draw zones using normalized coordinates.
4. Place fixed installations in the same coordinate system.
5. Visually verify alignment, scale, and zone membership.

Do not mix geographic coordinates with local floor coordinates.

Connectors

Connectors are separated into telemetry and catalog integrations. The recommended administrative workflow is:

1. Create an instance and select its provider.
2. Complete server-side configuration and credentials.
3. Test the connection using a minimal read operation.
4. Review the sanitized result.
5. Activate the connector.
6. Monitor its latest activity, errors, and pending volume.

Stored credentials are never displayed again. They must be changed through an explicit rotation procedure.

Telemetry, Positions, and History

Receiving telemetry does not guarantee that a position can be calculated. A position may remain unknown or have low confidence when anchors, coordinates, calibration data, or sufficient signal evidence are

unavailable.

When investigating an asset:

1. Review the device's latest reception time.
2. Confirm the active assignment between the device and asset.
3. Review available signal observations.
4. Verify the estimation algorithm, confidence, and accuracy.
5. Use historical data to distinguish an isolated reading from a consistent trajectory.

Alerts and Operations

Administrators and supervisors configure rules and authorized recipients. Every alert must be reviewed together with its evidence and observation time. An alert must not be interpreted as a guarantee of physical accuracy.

The operational health view reports failed or delayed telemetry, connector status, floor plans, pending scanners, and recent records. The scheduler must run every minute for deferred processing to advance.

Security and Recommended Practices

- Do not share accounts or connector credentials.
- Use unique passwords and authorized Microsoft sign-in where available.
- Verify the active organization before changing data.
- Do not copy payloads, sensitive locations, or secrets into unauthorized tickets or channels.
- Report errors with the date, screen, correlation identifier, and reproducible steps while redacting secrets.
- Request the least-privileged role required for the assigned work.

Support and Diagnostics

Before escalating an incident, record:

- the affected organization and user;
- date, time, and time zone;
- the affected asset, device, or connector;
- expected and observed results;
- the sanitized error message;
- visual evidence without secrets;
- operational scope and impact.

Technical procedures for diagnostics, backup, restoration, and continuity are provided in the deployment guide and operations runbook.

Executive Technical Summary

Purpose

LoraTrack is a web platform for inventory and asset location tracking in industrial indoor and outdoor environments. It manages products, physical assets, devices, floor plans, zones, connectors, IoT telemetry, and position estimates.

The application is implemented as a modular Laravel 12 monolith with Blade views, static CSS, and native JavaScript. It does not use a SPA framework or a Node.js frontend build pipeline.

Main Capabilities

- Multi-organization and multi-project operation through organizations and memberships.
- Product and SKU catalog management, including external catalog synchronization.
- Physical asset registration and time-bound device assignment.
- Device registry for beacons, scanners, gateways, LoRaWAN trackers, and access points.
- 2D floor plans, rectangular zones, installed anchors, real dimensions in meters, and basic 3D support.
- Telemetry ingestion from TTI, generic MQTT, and Meraki Location.
- BLE payload normalization with MAC and RSSI observations.
- RSSI multilateration and Kalman filtering for position estimates.
- Operational map, historical asset track view, and dashboard.
- Alerts for offline assets, low confidence, and zone events.
- Web mutation audit log and operational health view.

Technical Scope

LoraTrack becomes responsible once data is received from external providers. It does not implement a LoRaWAN network server and does not manage the LoRaWAN radio layer.

For LoRaWAN, The Things Industries manages devices, gateways, the network server, and webhook delivery. LoraTrack authenticates, deduplicates, stores, normalizes, and processes those events.

Architecture Summary

- Runtime: PHP 8.2+.
- Framework: Laravel 12.
- Persistence: SQL Server, MariaDB, or MySQL depending on the deployment baseline.
- Queue: Laravel queue driver configured by `QUEUE_CONNECTION`.
- UI: Blade, static CSS, and native JavaScript.

- Inbound integrations: routes under `/api/v1/ingest`.
- Heavy processing: Laravel jobs.
- Scheduling: Laravel scheduler through cron, Task Scheduler, or a service manager.

Critical Runtime Processes

Required background execution:

```
php artisan schedule:run
php artisan loratrack:mqtt-listen
```

Run `schedule:run` from cron every minute. The MQTT listener is only needed when MQTT connectors are enabled.

Relevant Design Controls

- Business entities are isolated by `organization_id`.
- Effective roles are derived from organization memberships.
- Webhook tokens are stored as connector credentials.
- Connector credentials are encrypted by Laravel.
- Floor plan files are served from authenticated private storage.
- External events are deduplicated and processed idempotently.
- Web mutations are audited.
- Connector errors are sanitized.
- PHPUnit tests cover integrations, positioning, roles, multi-tenancy, and storage behavior.

Known Limits

- RSSI accuracy depends on calibration, physical environment, and anchor quality.
- 2D positioning for mobile trackers requires at least three active, installed, non-collinear anchors on the same floor plan.
- The system must not be represented as certified metrology or a life-safety location system.
- Payload and observation retention must be formally defined per customer.
- Enterprise approval requires external evidence beyond the repository: access controls, backups, DRP, change management, infrastructure hardening, monitoring, and audit artifacts.

Solution Architecture

Architectural Style

LoraTrack is organized as a modular Laravel monolith. Domain modules live under `app/` and are separated by models, controllers, jobs, services, and connector adapters. The architecture avoids premature microservices and prioritizes local transactions, traceability, and operational simplicity.

Main Layers

Layer	Responsibility	Examples
Presentation	Blade views, forms, maps, progressive JavaScript	`resources/views`, `public/js`, `public/css`
HTTP	Authentication, authorization, validation, and responses	`app/Http/Controllers`, `app/Http/Requests`, `routes`
Application	Use cases, jobs, and commands	`app/Jobs`, `app/Console/Commands`
Domain	Models, positioning, telemetry, and connectors	`app/Models`, `app/Positioning`, `app/Telemetry`, `app/Connectors`
Persistence	Migrations, indexes, relationships, private storage	`database/migrations`, `storage/app/private`

Domain Modules

Catalog

Manages products, SKUs, and external references. Imports are performed through catalog connectors and normalized into internal models.

Relevant classes:

- `App\Models\Product`
- `App\Models\Sku`
- `App\Models\ExternalProductReference`
- `App\Connectors\CatalogProductImporter`
- `App\Jobs\SyncCatalogConnector`

Assets

Manages physical trackable instances. A product or SKU is not an asset; an asset has its own identity, photo, state, mobility, and current location.

Relevant classes:

- `App\Models\Asset`
- `App\Models\AssetDeviceAssignment`
- `App\Http\Controllers\AssetController`
- `App\Http\Controllers\AssetTrackController`

Devices

Manages registered hardware and installations. An installation has time validity and coordinates on a floor plan.

Relevant classes:

- `App\Models\Device`
- `App\Models\DeviceInstallation`
- `App\Http\Controllers\DeviceController`

Locations, Floor Plans, and Zones

Manages location hierarchy, raster plans, 3D models, zones, and anchors. Floor plans declare real width and height in meters.

Relevant classes:

- `App\Models\Location`
- `App\Models\FloorPlan`
- `App\Models\Zone`
- `App\Http\Controllers\FloorPlanController`
- `App\Positioning\ZoneClassifier`

Telemetry

Receives, deduplicates, normalizes, and processes external events. Raw events are stored in `telemetry_events.raw_payload`; normalized data is stored in `normalized_payload`.

Relevant classes:

- `App\Models\TelemetryEvent`
- `App\Models\SignalObservation`
- `App\Jobs\ProcessTtiUplink`
- `App\Jobs\ProcessMerakiLocationObservation`

- `App\Telemetry\AssetLastSeenUpdater`
- `App\Telemetry\TelemetryCounterUpdater`

Positioning

Converts RSSI observations into position estimates. Each estimate stores evidence, algorithm, version, confidence, and estimated accuracy.

Relevant classes:

- `App\Models\PositionEstimate`
- `App\Positioning\TelemetryPositioningService`
- `App\Positioning\RssiMultilateration`
- `App\Positioning\KalmanPositionFilter`
- `App\Positioning\BleObservationExtractor`

Connectors

Manages configured provider instances for telemetry and catalog synchronization. Non-secret configuration is stored separately from encrypted credentials.

Relevant classes:

- `App\Models\Connector`
- `App\Models\ConnectorActivityLog`
- `App\Connectors\ConnectorRegistry`
- `App\Connectors\ConnectorConnectionTester`

Identity and Security

Manages users, organizations, memberships, roles, local login, Microsoft OAuth, and server-side authorization.

Relevant classes:

- `App\Models\User`
- `App\Models\Organization`
- `App\Models\OrganizationMembership`
- `App\Enums\UserRole`
- `App\Http\Middleware\SetOrganizationContext`
- `App\Http\Middleware\EnsureUserHasPermission`

TTI Telemetry Flow

1. TTI sends `POST /api/v1/ingest/tti/{connector}` with a Bearer token.
2. `TtiWebhookController` validates size, token, minimum schema, and active connector status.

3. `external_event_id` is calculated from device, session, frame counter, and provider timestamp.
4. A `telemetry_events` row is created in `pending` state.
5. `schedule:run` invokes `loratrack:process-tti-uplinks`, which claims at most three pending uplinks per execution.
6. The command runs `ProcessTtiUplink` synchronously; it creates or updates the tracker device, normalizes the payload, extracts BLE observations, and updates `signal_observations` without requiring `queue:work`.
7. `AssetLastSeenUpdater` updates the assigned asset when applicable.
8. `TelemetryPositioningService` attempts to create `position_estimates`.
9. The event becomes `processed`, `failed`, or `ignored`.

Asset Track Flow

1. The user opens `/assets/{asset}/track`.
2. `AssetTrackController` selects floor plans with historical positions.
3. `asset-track.js` requests `/assets/{asset}/track/data`.
4. The endpoint returns `position_estimates` filtered by asset, floor plan, and time range.
5. The browser draws polylines, points, accuracy, and tooltips.

The track view does not read raw `telemetry_events`. If an uplink does not produce a `position_estimates` row, it does not appear as a track point.

External Dependencies

- Laravel Framework.
- Laravel Socialite and Socialite Microsoft provider.
- `php-mqtt/client` for MQTT.
- Vendored Select2 and jQuery for remote selectors.
- SQL Server, MariaDB, or MySQL depending on the deployment.
- External services: TTI, Meraki, SAP, Business Central, Shopify, and Odoo.

Evolution Principles

- Keep the internal domain independent from provider-specific payload formats.
- Process heavy integrations through queues.
- Store enough evidence for auditability and reproducibility.
- Do not mix geographic coordinates with local floor-plan coordinates.
- Prefer persisted counters and aggregates for operational screens.

Domain and Data Model

Core Concepts

Concept	Description
Organization	Business tenant or project. Isolates operational data.
User	Local or Microsoft-linked access account.
Membership	User-to-organization relationship with effective role.
Product	Commercial catalog definition.
SKU	Product code or catalog variant.
Asset	Physical trackable instance.
Device	Field hardware such as beacon, scanner, tracker, gateway, or AP.
Assignment	Time-bound asset-to-device relationship.
Installation	Time-bound fixed device placement on a floor plan.
Telemetry	External event received from TTI, Meraki, MQTT, or another provider.
Signal observation	MAC/RSSI observation detected by a receiver.
Position estimate	Calculated result with algorithm, evidence, and accuracy.
Floor plan	Raster or 3D representation of a physical space with real dimensions.
Zone	Region inside a floor plan.
Connector	Configured external provider instance.

Relevant Tables

Organizations and Identity

- `organizations`: name, slug, branding, operational settings.
- `organization_memberships`: user, organization, and role.
- `organization_invitations`: invitations with token and expiration.
- `users`: account, email, password, and `microsoft_id`.

Catalog

- `products`: normalized product.
- `skus`: SKU code and product reference.
- `external_product_references`: provider/external ID mapping.

Assets and Devices

- `assets`: physical asset, tag, name, mobility, status, photo, `last_seen_at`.
- `devices`: hardware identity, type, status, metadata, `last_seen_at`.
- `asset_device_assignments`: time-bound asset-device assignment and tracking strategy.
- `device_installations`: fixed device placement with coordinates, floor plan, and RSSI parameters.

Locations and Floor Plans

- `locations`: sites, buildings, floors, or other hierarchy levels.
- `floor_plans`: private files, real dimensions, 2D/3D configuration.
- `zones`: rectangles in normalized coordinates.

Telemetry and Positioning

- `telemetry_events`: external event, raw payload, normalized payload, processing status.
- `signal_observations`: MAC/RSSI observations associated with an event.
- `position_estimates`: calculated asset positions tied to telemetry events.
- `calibration_runs`: RSSI calibration history.

Connectors and Audit

- `connectors`: provider instance, status, configuration, encrypted credentials, counters.
- `connector_activity_logs`: connector activity log.
- `audit_logs`: web mutation audit trail.

Alerts

- `alert_settings`: organization-level alert settings.
- `alerts`: alert events.
- `alert_rules`: configurable alert rules.
- `zone_alert_rules`: zone-specific rules.
- `zone_presence_states`: current zone presence state by asset.

Multi-Tenancy

Business entities use `organization_id` and the `BelongsToOrganization` trait. The active context is resolved through `SetOrganizationContext`.

Expected rules:

- A user only sees data from the active organization.
- Roles are derived from memberships, not from a global role.
- Routes must enforce server-side authorization.
- `unique` and `exists` validation rules must be tenant-aware where applicable.
- Jobs and webhooks must set organization context before operating.

Time and Traceability

Telemetry stores separate timestamps:

- `observed_at`: provider or device time, normalized.
- `received_at`: time when LoraTrack received the event.
- `processed_at`: time when the job completed.

Positions store:

- `calculated_at`: calculation time.
- `telemetry_event_id`: source event.
- `evidence`: anchors, RSSI, estimated distances, and residuals.
- `algorithm` and `algorithm_version`.
- `confidence` and `accuracy_meters`.

Event Identity

TTI events are deduplicated using a hash built from:

- `end_device_ids.dev_eui`
- `end_device_ids.device_id`
- `uplink_message.session_key_id`
- `uplink_message.f_cnt`
- `uplink_message.received_at` or `received_at`

This avoids reprocessing provider retries without collapsing successive uplinks.

Data Governance Notes

- Define retention for `raw_payload` formally.
- Treat floor plans and asset locations as sensitive site information.
- Avoid exposing raw payloads in logs or broad UI views.
- Define contractual/legal basis for location data.
- Define export and deletion procedures for contract termination.

Telemetry and Positioning

Objective

Convert external provider events into normalized observations and, when evidence is sufficient, estimate asset positions on floor plans with real-world dimensions.

Telemetry States

`telemetry_events.processing_status` may be:

- `pending`: event received and not yet processed.
- `processed`: job completed successfully.
- `failed`: job failed and stores a sanitized error.
- `ignored`: event intentionally not processed, for example because the connector was disabled.

TTI Ingestion

Endpoint:

```
POST /api/v1/ingest/tti/{connector}
Authorization: Bearer {token}
Content-Type: application/json
```

Main validations:

- Maximum payload size is 1 MB.
- Connector must be active and of provider `TTI Webhook`.
- Bearer token must match the configured credential.
- `end_device_ids` and `uplink_message` must be present.
- `uplink_message.received_at` or `received_at` is used as provider time when available.

The endpoint returns `202 Accepted` after durable persistence. Processing is deferred to the Laravel scheduler and is not dispatched to the queue worker.

TTI Processing

Scheduled command: `loratrack:process-tti-uplinks`. It processes at most three pending TTI events per execution and invokes `App\Jobs\ProcessTtiUplink` synchronously for the existing idempotent processing logic.

Responsibilities:

1. Set organization context.
2. Skip events already marked as `processed`.
3. Mark queued events as `ignored` if the connector was disabled.
4. Create or update the tracker device.
5. Apply a decoder profile when one matches.
6. Store `normalized_payload`.
7. Extract BLE MAC/RSSI observations.
8. Update `last_seen_at` for the assigned asset.
9. Run positioning.
10. Mark the event as processed or failed.

BLE Extraction

Class: `App\Positioning\BleObservationExtractor`

Accepted observation fields:

- MAC: `mac`, `mac_address`, `address`, `beacon_mac`.
- RSSI: `rssi`, `signal`, `signal_strength`.

MAC addresses are normalized to uppercase hexadecimal without separators. RSSI must be numeric, negative, and not less than -127 dBm.

Tracking Strategies

Mobile Beacon, Fixed Scanners

A BLE beacon is assigned to a mobile asset. Fixed scanners detect its MAC. The system groups recent observations by receiver and calculates position against `scanner` installations.

Strategy: `mobile_beacon_fixed_scanners`

Fixed Beacons, Mobile Tracker

A LoRaWAN tracker is assigned to a mobile asset. The tracker reports detected fixed beacons. The system calculates position using `beacon` installations.

Strategy: `fixed_beacons_mobile_tracker`

Conditions Required to Generate a Position

An event creates a `position_estimates` row only when:

- the event has `device_id`;
- the event has signal observations;
- the device is actively assigned to an asset;
- the tracking strategy matches;
- at least three active anchors of the correct type exist;
- anchors have `x/y` coordinates;
- anchors belong to the same `floor_plan_id`;
- geometry is not degenerate for multilateration.

If any condition is missing, the event may still be `processed` without creating a position.

Algorithm

RSSI positioning uses:

- `RssiMultilateration`: approximates position from RSSI measurements.
- `KalmanPositionFilter`: smooths successive positions.
- `ZoneClassifier`: assigns a position to rectangular zones.

Each `position_estimates` row stores:

- `raw_x`, `raw_y`: pre-filter result.
- `x`, `y`: filtered result.
- `confidence`.
- `accuracy_meters`.
- `evidence`: anchors, RSSI, distances, and residuals.
- `filter_state`.

Asset Track View

The asset track screen queries only `position_estimates`. It does not draw raw `telemetry_events`.

Applied filters:

- asset;
- selected floor plan;
- time range based on `calculated_at`;
- optional `after` timestamp for live updates.

If new uplinks arrive but no new track point appears, check:

1. `telemetry_events.processing_status`.
2. number of `signal_observations`.
3. asset-device assignment.

4. active and installed beacons/scanners.
5. selected floor plan and time range.
6. `position_estimates` for the telemetry event.

Operational Diagnostic SQL

Recent events:

```
select id, device_id, observed_at, received_at, processed_at,
       processing_status, processing_error
from telemetry_events
order by received_at desc
limit 20;
```

Observations by event:

```
select telemetry_event_id, count(*) as signals
from signal_observations
group by telemetry_event_id
order by max(observed_at) desc
limit 20;
```

Recent positions:

```
select id, asset_id, telemetry_event_id, floor_plan_id,
       calculated_at, x, y, confidence, accuracy_meters
from position_estimates
order by calculated_at desc
limit 20;
```

Position Quality

RSSI location is an operational estimate, not certified metrology. Quality depends on:

- anchor calibration;
- multipath and site materials;
- antenna height and orientation;
- anchor density and distribution;
- firmware and payload format;
- uplink frequency and time window.

Retention

Telemetry storage management commands:

- `loratrack:manage-telemetry-storage`
- `loratrack:prune-meraki-history`

The exact retention policy must be defined by customer contract and risk assessment.

External Integrations and Contracts

Principles

- Each external provider is translated into internal models.
- Domain logic must not depend on provider SDKs or payload field names.
- Heavy connector work runs through queues.
- Credentials are encrypted.
- User-visible errors are sanitized.
- Tests should use sanitized fixtures.

Connector Types

Telemetry

- TTI Webhook.
- Generic MQTT.
- Meraki Location.

Catalog

- SAP S/4HANA.
- Microsoft Dynamics 365 Business Central.
- Shopify.
- Odoo.
- CSV.

Connector Model

Table: `connectors`

Conceptual fields:

- organization;
- name;
- kind;
- provider;
- status;
- non-secret configuration;
- encrypted credentials;
- contract version;

- cursor or checkpoint;
- last activity;
- last success;
- sanitized last error;
- telemetry counters.

If a connector is disabled, queued telemetry is not processed. The job marks those events as `ignored`.

TTI Webhook

TTI responsibilities:

- LoRaWAN network server;
- gateways;
- devices;
- session and frame counters;
- webhook delivery.

LoraTrack responsibilities:

- authenticate webhook;
- deduplicate events;
- store raw event;
- normalize payload;
- extract BLE observations;
- calculate position when possible.

Minimum contract:

```
{
  "end_device_ids": {
    "device_id": "tracker-01",
    "dev_eui": "0011223344556677"
  },
  "received_at": "2026-07-10T11:24:49Z",
  "uplink_message": {
    "f_cnt": 42,
    "f_port": 1,
    "received_at": "2026-07-10T11:24:49Z",
    "decoded_payload": {
      "beacons": [
        {"mac": "AA:BB:CC:DD:EE:01", "rssi": -76}
      ]
    }
  }
}
```

`uplink_message.received_at` is preferred for identity and `observed_at` when present.

MQTT

Command:

```
php artisan loratrack:mqtt-listen
```

The listener connects to the broker configured in the connector. It must be supervised by systemd, Supervisor, Task Scheduler, or an approved equivalent.

MQTT does not bypass queue processing. Messages are converted to events and follow the normal telemetry flow.

Meraki Location

Routes:

```
GET  /api/v1/ingest/meraki/{connector}
POST /api/v1/ingest/meraki/{connector}
```

The integration includes receiver validation, normalization, access point registration, and Meraki-to-internal floor plan mapping.

SAP S/4HANA

The connector targets the Product Master API. Configuration includes:

- base URL;
- base path;
- authentication;
- timeout;
- cursor or incremental strategy when supported.

The adapter must not leak SAP entities such as `A_Product` outside the connector. Normalization produces internal products and SKUs.

Business Central, Shopify, Odoo, and CSV

These connectors import products and normalize them into internal catalog records. They must respect:

- pagination;
- rate limits;
- idempotency;
- cursors/checkpoints;
- no deletion of local products because of partial provider responses.

CSV is a controlled manual import mechanism.

Connection Testing

Connection tests must:

- use strict timeouts;
- not perform implicit imports;
- not expose credentials;
- store sanitized errors;
- log connector activity.

Versioning

Each integration should document:

- external API version;
- review date;
- endpoints used;
- required permissions;
- pagination policy;

- limits and rate limits;
- error format;
- sanitized fixture.

Connector Onboarding Checklist

- [ ] Correct provider and kind.
- [ ] Non-secret configuration validated.
- [ ] Credentials loaded and encrypted.
- [ ] Connection test successful.
- [ ] Connector active.
- [ ] Queue worker running.
- [ ] Scheduler running when required.
- [ ] Logs do not contain secrets.
- [ ] Test fixture available.
- [ ] Rotation procedure documented.

Internal and External API Contracts

Scope

This document summarizes relevant HTTP routes for integrations, operations, and visualization. It is not a formal OpenAPI specification, but it provides a review baseline for engineering teams.

Conventions

- Web routes require an authenticated session unless stated otherwise.
- Ingestion routes use `/api/v1`.
- Exposed identifiers are generally ULIDs.
- Error responses follow Laravel behavior unless explicitly customized.
- Examples are sanitized.

TTI Ingestion

```
POST /api/v1/ingest/tti/{connector}
```

Headers:

```
Authorization: Bearer {webhook_token}
Content-Type: application/json
Accept: application/json
```

Minimum payload:

```
{
  "end_device_ids": {
    "device_id": "tracker-01",
    "dev_eui": "0011223344556677"
  },
  "uplink_message": {
    "f_port": 1,
    "f_cnt": 42,
    "received_at": "2026-07-10T11:24:49Z",
    "decoded_payload": {
      "beacons": [
        {"mac": "AA:BB:CC:DD:EE:01", "rssi": -76},
        {"mac": "AA:BB:CC:DD:EE:02", "rssi": -78},
        {"mac": "AA:BB:CC:DD:EE:03", "rssi": -80}
      ]
    }
  }
}
```

Accepted response:

```
{
  "accepted": true,
  "duplicate": false,
  "event_id": "01...",
  "request_id": "..."
}
```

Status codes:

- `202`: accepted.
- `401`: invalid token.
- `404`: connector not found, inactive, or wrong provider.
- `413`: payload larger than 1 MB.
- `422`: validation failure.
- `429`: throttled.

Meraki Ingestion

Receiver validation:

```
GET /api/v1/ingest/meraki/{connector}
```

Reception:

```
POST /api/v1/ingest/meraki/{connector}
```

The connector must be active and configured. Internal normalization produces `telemetry_events`, `signal_observations`, access point devices, and positions when mapping is sufficient.

Authenticated Meraki POST requests are stored in a durable, idempotent inbox and receive `200 OK` before normalization. The shared secret is removed before persistence. `schedule:run` processes pending inbox records, normalizes and deduplicates observations, creates telemetry events, records connector activity, and dispatches downstream observation processing.

Rejected Meraki POST requests are recorded separately from accepted telemetry. LoraTrack retains only the 10 most recent rejections per connector with the HTTP status, sanitized reason, declared version/type, request ID, and a keyed hash of the source IP. Shared secrets and complete request payloads are never stored in this diagnostic history. These records do not increment the failed telemetry counter because no `telemetry_event` was accepted.

The connector detail counters link to filtered accepted telemetry (`received`, `processed`, `pending`, or `failed`) or to the separate rejected-request history.

Map Data

```
GET /map/{floorPlan}/data
```

Authorization:

- authenticated session;
- `maps.view` permission.

Conceptual response:

```
{
  "anchors": [
    {
      "id": "01...",
      "name": "Beacon A",
      "identifier": "AABBCCDDEE01",
      "type": "beacon",
      "x": 0.1,
      "y": 0.2
    }
  ],
  "positions": [
    {
      "asset_id": "01...",
      "name": "Pump 01",
      "x": 0.5,
      "y": 0.4,
      "x_meters": 5.0,
      "y_meters": 4.0,
      "confidence": 0.91,
      "accuracy_meters": 1.2,
      "calculated_at": "2026-07-10T11:24:50Z",
      "evidence": []
    }
  ]
}
```

Asset Track

View:

```
GET /assets/{asset}/track
```

Data:

```
GET /assets/{asset}/track/data?floor_plan_id={id}&range=24h
```

Parameters:

- `floor_plan_id`: optional.
- `range`: `1h`, `24h`, `7d`, `30d`.
- `from`: optional date.

- `to`: optional date.
- `after`: optional incremental update date.

The response contains asset metadata, floor plan metadata, and historical `position_estimates`.

Connector Administration

Admin web routes:

- `GET /connectors`
- `GET /connectors/{connector}`
- `GET /connectors/create/{provider}`
- `POST /connectors`
- `POST /connectors/{connector}/test`
- `POST /connectors/{connector}/toggle`
- `POST /connectors/{connector}/rotate-webhook-token`
- `POST /connectors/{connector}/sync`
- `POST /connectors/{connector}/csv`

Secrets must not be returned in full to the browser after storage.

Floor Plans, Zones, and Installations

Relevant routes:

- `GET /floor-plans`
- `GET /floor-plans/{floorPlan}/file`
- `GET /floor-plans/{floorPlan}/model`
- `POST /floor-plans`
- `PUT /floor-plans/{floorPlan}`
- `POST /floor-plans/{floorPlan}/zones`
- `POST /floor-plans/{floorPlan}/installations`
- `PUT /installations/{deviceInstallation}`

Mutation permission:

```
plans.manage
```

Assets

Relevant routes:

- `GET /assets`

- `GET /assets/{asset}/photo`
- `POST /assets`
- `PUT /assets/{asset}`
- `POST /assets/{asset}/assignments`
- `DELETE /asset-assignments/{assignment}`
- `POST /assets/{asset}/refresh-position`

Mutation permission:

```
assets.manage
```

Error Observability

For investigations, correlate:

- `X-Request-ID` when present;
- `telemetry_events.id`;
- `connectors.id`;
- `jobs.uuid` when applicable;
- `observed_at`, `received_at`, and `processed_at`;
- Laravel logs.

Future OpenAPI Requirements

- Versioned schemas per endpoint.
- Stable error code catalog.
- Pagination and filter documentation.
- Permission documentation per route.
- Sanitized examples per provider.
- Outbound webhook contracts if added.

Security, Identity, and Tenant Isolation

Identity Model

LoraTrack supports:

- local email/password login;
- Microsoft OAuth/OpenID Connect through Laravel Socialite;
- public organization registration;
- invitations to existing organizations.

A user account does not define effective authorization by itself. Effective authorization depends on the user's membership in the active organization.

Roles

Roles defined in `App\Enums\UserRole`:

Role	Capabilities
admin	Full access, connectors, users, and security administration.
engineer	Floor plans, anchors, calibration, decoders, and technical diagnostics.
supervisor	Assets, alerts, and operational supervision.
operator	Daily asset registration, assignment, and tracking.
viewer	Read-only access to products, assets, plans, and maps.

Permissions are enforced by middleware and controllers. Hiding a menu item is not a security control.

Multi-Tenancy

Isolation is based on:

- `organization_id` on business entities;
- `BelongsToOrganization` trait;
- `OrganizationContext`;
- `SetOrganizationContext` middleware;
- tenant-filtered route model binding and queries;
- tenant-aware validation where applicable.

Cross-organization access should return 404 or 403 depending on context.

Authentication

Local Login

Laravel provides:

- password hashing;
- sessions;
- CSRF protection;
- session regeneration at login;
- login throttling.

Microsoft

Required environment variables:

```
MICROSOFT_CLIENT_ID=
MICROSOFT_CLIENT_SECRET=
MICROSOFT_TENANT_ID=
MICROSOFT_REDIRECT_URI=
```

The link uses stable `microsoft_id`. Accounts must not be automatically merged only by email unless a formal policy approves it.

Secrets

Do not version:

- `.env`;
- connector credentials;
- TTI tokens;
- Microsoft secrets;
- database dumps;
- real payloads;
- customer floor plans;
- SSH keys.

Connector credentials must be encrypted with Laravel capabilities. The UI must not re-expose full secrets after saving them.

Private Files

Floor plans are stored under `storage/app/private` and served through authenticated routes. Do not expose floor plans through a public storage symlink.

Webhooks

Current controls:

- payload size limit;
- Bearer token for TTI;
- active connector required;
- telemetry route throttling;
- deduplication;
- asynchronous processing;
- sanitized errors.

Recommended enterprise controls:

- mandatory TLS;
- scheduled token rotation;
- provider IP allowlisting where available;
- WAF or reverse proxy rate and size limits;
- `X-Request-ID` logging;
- monitoring of retries and failures.

Audit

Web mutations generate records in `audit_logs` with user, route, result, and request ID. Audit records must not include secrets or full sensitive payloads.

Security Headers

`SecurityHeaders` middleware should remain enabled for web responses. Reverse proxies must be reviewed to avoid weakening or duplicating headers incorrectly.

Data Classification

Potentially sensitive data:

- site plans and facility images;
- asset positions;
- device identifiers;

- telemetry payloads;
- users, emails, and memberships;
- connector tokens and credentials.

Treat these as sensitive operational customer information.

Controls Outside the Repository

Enterprise approval commonly requires external evidence:

- server hardening;
- vulnerability management;
- backup and restore testing;
- disaster recovery plan;
- information asset inventory;
- access matrix;
- periodic access review;
- change management;
- environment segregation;
- centralized logging;
- incident response and monitoring;
- support agreements and SLA.

Operations, Monitoring, and Runbooks

Objective

Define the minimum tasks required to operate LoraTrack in an enterprise environment and diagnose common failures.

Required Processes

Scheduler

Run every minute:

```
php artisan schedule:run
```

The scheduler is the only required background executor. It processes Meraki batches and observations, TTI/MQTT telemetry, and requested catalog synchronizations without Laravel Queue. TTI and MQTT commands process at most three pending events per execution.

Scheduled tasks:

- `loratrack:evaluate-alerts` every 10 minutes.
- `loratrack:manage-telemetry-storage` hourly.
- `loratrack:prune-meraki-history` hourly.

MQTT Listener

When MQTT connectors are configured:

```
php artisan loratrack:mqtt-listen
```

The listener must be supervised and restarted on failure.

Operational Health View

Route:

```
/operations/health
```

Review:

- stuck telemetry;
- connector errors;
- private storage state;
- anchors by floor plan;
- recent audit records.

Logs

Laravel log:

```
storage/logs/laravel.log
```

Scheduler cron output example:

```
php artisan schedule:run >> storage/logs/schedule.log 2>&1
```

Do not enable tracing that prints credentials.

Runbook: Telemetry Stays Pending

1. Confirm scheduler execution:

```
php artisan schedule:run -v
```

2. Review recent events:

```
select id, connector_id, device_id, processing_status, processing_error,
       observed_at, received_at, processed_at
from telemetry_events
order by received_at desc
limit 20;
```

3. Check database connection limits if `max_user_connections` appears.
4. Confirm there is only one cron entry invoking the scheduler each minute.

Runbook: TTI Arrives but Asset Time Does Not Update

1. Check recent event identity and timestamps.
2. Check event processing status.
3. Check active asset-device assignment.

4. Check that the event has signal observations.
5. Check whether a position estimate was generated.

Useful SQL:

```
select id, device_id, observed_at, received_at, processing_status, processing_error
from telemetry_events
order by received_at desc
limit 10;
```

Runbook: Track View Shows Fewer Points Than Expected

The track view shows `position_estimates`, not raw `telemetry_events`.

Review:

```
select id, asset_id, telemetry_event_id, floor_plan_id, calculated_at, x, y
from position_estimates
where asset_id = '{asset_id}'
order by calculated_at desc;
```

If telemetry exists without a position, check:

- fewer than three valid MAC/RSSI observations;
- anchors not installed;
- inactive anchors;
- anchors on different floor plans;
- expired assignment;
- failed event;
- floor plan filter in the UI.

Runbook: Disabled Connector

A disabled connector does not process telemetry. Queued events are marked `ignored` when taken by the job.

To resume:

1. Activate connector.
2. Send a new payload or requeue events according to policy.
3. Confirm `last_activity_at` and `last_success_at`.

Runbook: Database Connection Limit Exhausted

Symptom:

```
SQLSTATE[42000] [1226] User has exceeded the max_user_connections resource
```

Actions:

- remove duplicate scheduler cron entries;
- review cron overlap;
- review persistent connections;
- increase database limits if load justifies it.

Monthly Maintenance Checklist

- review users and roles;
- review active connectors and tokens;
- test backup restoration;
- review `telemetry_events` growth;
- review failed jobs;
- validate SMTP alerts;
- review recurring log errors;
- run `composer audit` during a controlled window;
- update change documentation.

Escalation Data

For level 2/3 support collect:

- exact timestamp;
- organization;
- connector;
- device identifier;
- telemetry event ID;
- asset ID;
- position estimate ID when present;
- sanitized log excerpt;
- deployed release version;
- recent configuration changes.

Field Commissioning Guide

Objective

Define a baseline procedure for installing, validating, and calibrating LoraTrack in an industrial facility.

Suggested Roles

Role	Responsibility
Project lead	Scope, work windows, and acceptance.
OT/IT engineering	Network, access, servers, security, and integrations.
RF/IoT specialist	Devices, beacons, trackers, and gateways.
LoraTrack administrator	Organizations, users, connectors, and configuration.
Customer operations	Use case validation and acceptance criteria.

Prerequisites

- Current floor plan.
- Real dimensions in meters.
- Asset inventory.
- Device inventory.
- TTI, MQTT, or Meraki connectivity as required.
- LoraTrack environment access.
- Approved credentials and tokens.
- Authorized installation window.
- Defined success criteria.

Step 1: Environment

1. Confirm the deployed release version.
2. Confirm `APP_DEBUG=false`.
3. Confirm scheduler operation every minute.
4. Confirm scheduled telemetry processing.
5. Confirm SMTP if alerts are used.
6. Confirm backups.

7. Confirm basic monitoring.

Step 2: Organization and Users

1. Create organization.
2. Create administrator.
3. Configure branding if required.
4. Invite users.
5. Assign roles by responsibility.
6. Validate access with a non-admin user.

Step 3: Floor Plans

1. Create location.
2. Upload raster plan or preview image.
3. Register real width and height in meters.
4. Verify orientation.
5. Draw operational zones.
6. Store evidence of dimensions used.

Step 4: Fixed Devices

1. Register beacons or scanners.
2. Install physically.
3. Register coordinates on the floor plan.
4. Confirm correct type: `beacon` or `scanner`.
5. Confirm status `active`.
6. Register initial RSSI parameters: - reference RSSI; - path loss exponent.

Step 5: Connectors

TTI

1. Create TTI connector.
2. Generate a long token.
3. Activate connector.
4. Configure webhook in TTI.
5. Send a test payload.
6. Confirm event is `processed`.
7. Confirm observations in `signal_observations`.

MQTT

1. Create MQTT connector.
2. Validate host, port, TLS, and credentials.
3. Run listener.
4. Confirm message reception.

Catalog

1. Create connector.
2. Test connection.
3. Run synchronization.
4. Validate imported products and SKUs.

Step 6: Assets and Assignments

1. Create or import assets.
2. Register trackers or mobile beacons.
3. Assign device to asset.
4. Select strategy: - `fixed_beacons_mobile_tracker`; - `mobile_beacon_fixed_scanners`.
5. Validate start date.
6. Avoid duplicate active assignments.

Step 7: Position Validation

1. Place asset at a known point.
2. Wait for or force an uplink.
3. Confirm `telemetry_events`.
4. Confirm at least three valid observations.
5. Confirm `position_estimates`.
6. Compare calculated position with real point.
7. Record observed error.
8. Repeat in multiple points.

Step 8: Calibration

Use the floor plan calibration workbench:

1. Select strategy.
2. Enter real X/Y point.
3. Enter median RSSI per anchor.
4. Review RMSE and residuals.

5. Adjust reference RSSI and path loss exponent.
6. Apply only if the estimate improves.
7. Keep calibration history.

Step 9: Acceptance Criteria

Examples:

- telemetry processed under N seconds;
- percentage of uplinks with valid position;
- median error below N meters in pilot zone;
- map updates within expected interval;
- track view shows expected history;
- SMTP alerts reach recipients;
- dashboards load within target time;
- users only see their organization.

Step 10: Operational Handover

Deliver:

- customer-managed credentials;
- connector list;
- token rotation owners;
- device inventory;
- location and floor plan map;
- calibration parameters;
- runbooks;
- support procedure;
- accepted risks.

Commissioning Evidence

Store:

- date and time;
- release version;
- participants;
- floor plan used;
- test points;
- position results;
- detected errors;

- corrective actions;
- counterpart approval or sign-off.

TTI Integration

LoraTrack receives uplinks from The Things Stack through HTTPS. TTI remains responsible for the LoRaWAN network, gateways, devices, and delivery.

Inbound Contract

- Endpoint: `POST /api/v1/ingest/tti/{connector}`
- Authentication: `Authorization: Bearer <token>`
- Accepted response: HTTP 202
- Idempotent identity: stable hash of device, session, frame counter, and TTI timestamp

LoraTrack stores `raw_payload`, receive time, and the normalized version. The job extracts `decoded_payload`, `frm_payload`, `rx_metadata`, frame counter, port, and device information.

Reference: https://www.thethingsindustries.com/docs/integrations/webhooks/

SAP S/4HANA Integration

The initial connector consumes Product Master through OData (`API_PRODUCT_SRV`). The base URL and path are configurable to support different deployments.

Normalization

SAP	LoraTrack
`Product`	external reference and SKU code
`ProductDescription`	product/SKU name
`BaseUnit`	base unit
`ProductType`	attribute
`ProductGroup`	attribute

Codes are stored as text and preserve leading zeroes. Each reference is unique by connector and SAP identifier. A checksum avoids unnecessary writes.

Product Master does not represent stock by plant or warehouse. That capability requires a separate API and synchronizer.

Reference: https://api.sap.com/api/API_PRODUCT_SRV/overview